# Day 1
# Onboarding

Claude Code와 대화하면서 익힌다

AI Native Camp 1기

2026. 2. 14 I

# 오늘의 여정

**0**

**Setup**

설치 + 첫 대화

→

**1**

**Experience**

데모 3가지 체험

→

**2**

**Why**

왜 CLI? 왜 터미널?

→

**3**

**What**

7개 핵심 기능

→

**4**

**Basics**

CLI + git + GitHub

Setup

# 설치 + 첫 대화 + CLAUDE.md

Mac / Linux

**curl -fsSL https://claude.ai/install.sh | bash**

Windows (PowerShell)

**irm https://claude.ai/install.ps1 | iex**

Node.js 불필요. 한 줄이면 끝. 자동 업데이트 내장.

### ① 설치

위 명령어 실행

### ② 첫 실행

터미널에 claude 입력

### ③ Output Style

/output-style → Explanatory 선택

### ④ 자유 대화

"안녕, 나는 [이름]이고 [직업]이야"

Experience

# 7일 후 모습을 먼저 체험한다

외우지 않는다. 느끼기만 한다.

| Before (지금) | After (7일 후) |
|---|---|
| 슬랙·캘린더·노션 하나씩 열어서 30분 | 한 문장으로 자동 발송 |
| "앱 만들어줘" → 뭘 만들지 모름 | 모호한 요청 → Claude가 질문으로 명확화 |
| 기능 외우려고 문서 뒤지기 | 모르면 Claude에게 물어보기 |

Experience

# 3가지 데모

## 데모 1

**/weekly-sync**

강사 시연

한 마디로 Slack 수집

→ git 확인 → 문서 생성

7일 후에는 이걸

직접 만들 수 있다

## 데모 2

**AskUserQuestion**

직접 실행

모호하게 요청

→ Claude가 추측

AskUserQuestion 활용

→ Claude가 질문으로 명확화

## 데모 3

**Claude에게 물어보기**

직접 실행

"MCP가 뭐야?"

기능을 다 외울 필요 없다

모르면 물어보면 된다

Why

# 왜 CLI인가?

## GUI

사용자 → [버튼] [버튼] [버튼] → 제한된 기능

## CLI

사용자 → 자유로운 명령어 ————→ 컴퓨터의 모든 기능

AI 에이전트 → CLI ————————→ 소프트웨어 직접 제어

**CLI는 컴퓨터의 모든 일을 할 수 있다**

Why

# 왜 CoWork가 아니라 터미널인가?

Cursor, CoWork 같은 도구

→ 정해진 UI 안에서의 생산성

터미널 Claude Code

→ **코딩이라는 새로운 차원의 일**

## 모두가 코딩으로 새로운 차원의 일을 하게 된다

토큰을 많이 쓸 수 있다 = 엄청나게 많은 노동력을 가진 사람이 된다

Block 3

# What
# 7개 핵심 기능


| | |
|---|---|
| 3-1 | CLAUDE.md |
| 3-2 | Skill |
| 3-3 | MCP |
| 3-4 | Subagent |
| 3-5 | Agent Teams |
| 3-6 | Hook |
| 3-7 | Plugin |

3-1 CLAUDE.md

# Claude가 매번 읽는 규칙서

| 항목 | 내용 |
|---|---|
| 근본 원리 | 시스템 프롬프트 — AI가 대화를 시작할 때 가장 먼저 읽는 지시문 |
| 비유 | 팀 위키 — Claude가 매 대화 시작할 때 읽는 규칙서 |
| 예시 | "항상 존댓말로", "표로 정리해줘", "내 이름은 OOO" |

세션 시작 → CLAUDE.md (영구 기억) → 시스템 프롬프트 → 규칙을 아는 상태로 대화

**실습: /init (자동 생성) I /memory (개인 기억 저장)**

3-2 Skill

# 반복 업무를 레시피로 저장

## 근본 원리: 점진적 로딩

CLAUDE.md는 항상 전부 읽히지만

**Skill은 필요할 때만 꺼내 읽힌다**

컨텍스트 윈도우는 유한하니까

CLAUDE.md ———————————— 항상 로딩 (매 세션)

Skill A ———┐ "/sync" 호출 시

Skill B ————————┐ 자동 매칭 시

Skill C ————————————┐ 필요할 때만 로딩

## Skill 폴더 구조

```
my-skill/
├── SKILL.md       # 필수: 본체
├── scripts/       # 선택: 코드
├── references/    # 선택: 참고
└── assets/        # 선택: 리소스
```

3-3 MCP

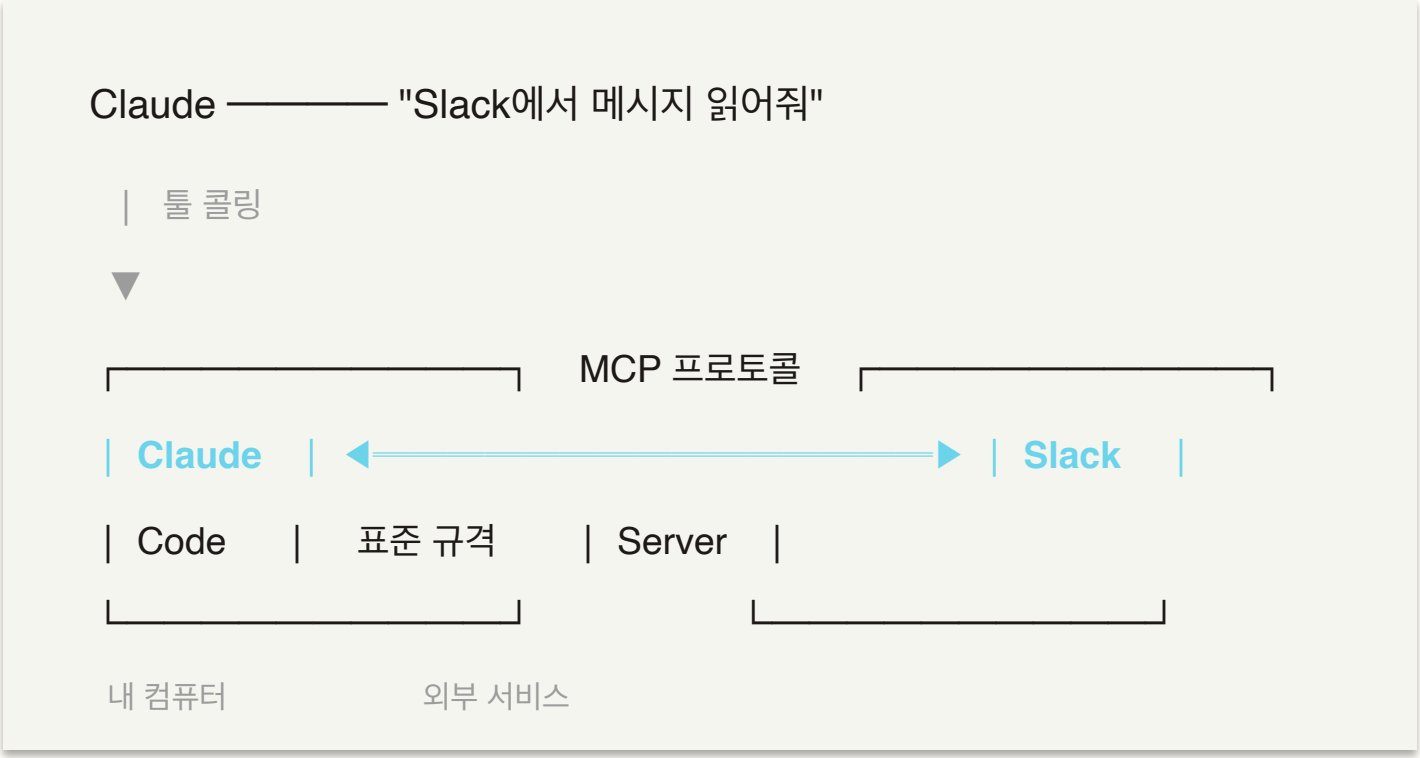
# 외부 도구를 연결하는 표준 프로토콜

## 근본 원리: 툴 콜링 (Tool Calling)

AI가 텍스트만 생성하는 게 아니라

**"이 함수를 이 파라미터로 호출해"**

MCP는 이걸 외부 서비스까지 확장하는 오픈 표준



3-4 Subagent

# 독립된 공간에서 전담 처리

## 근본 원리: 프로세스 격리 + Blank Slate

메인 대화의 컨텍스트를 물려받지 않고

**빈 상태에서 작업 → 요약만 돌려준다**

```
┌── 메인 Claude ──────────────────────────────────────────┐
│  대화 컨텍스트: [A, B, C, D, E...]                       │
│  "PDF 분석해줘" ──┐                                      │
│   ┌── Subagent ──────────────────────┐                   │
│   │ 컨텍스트: [blank slate]          │                   │
│   │ 결과: 요약 3줄 ──────────────▶   │ 전달              │
│   └──────────────────────────────────┘                   │
└──────────────────────────────────────────────────────────┘
```

# 쉬어가기

여기까지 핵심 4개: CLAUDE.md, Skill, MCP, Subagent

## 터미널 추천

Ghostty — 빠르고 가벼움, GPU 가속

iTerm2 — macOS 대표, 풍부한 기능

WezTerm — 크로스플랫폼, Lua 설정

## Status Line 설정

Claude Code에 이렇게 말하면 끝:

**/statusline 모델 이름과**

**컨텍스트 사용률을 보여줘**

결과: [Opus] ▦▦▦▦▦▦▦▦▦▦ 25%

3-5 Agent Teams

# 여러 Agent가 동시에 협업

## Subagent vs Agent Teams

Subagent: 부하 직원 (1:1 위임, 보고만)

**Teams: 프로젝트 팀 (팀원끼리 소통 + 공유 태스크)**

```
┌── 리더 ──────────────────────────────────────────────────────────────┐
│   ┌ Agent A ┐   ┌ Agent B ┐   ┌ Agent C ┐      │
│   │ 시장조사 │   │ 보고서  │   │ 발표자료 │      │
│   └─────┬───┘   └─────┬───┘   └─────┬───┘      │
│         └──────── 메시지 + 공유 태스크 ────────┘      │
└──────────────────────────────────────────────────────────────────────┘
```

3-6 Hook

# 특정 이벤트 시 자동 실행

## 근본 원리: 결정론적 프로그래밍

AI는 "대부분" 맞지만 100%는 아니다. Hook은 코드가 실행되므로 100% 확실하게 동작한다.

### AI (확률적)

"파일 저장 후 포맷팅 해줘"

|

▼

대부분 함. 가끔 까먹음

### Hook (결정론적)

파일 저장 이벤트

|

▼

**prettier --write . ← 100% 실행**

3-7 Plugin

# 모든 기능을 묶어서 팀에 공유

## 근본 원리: 패키징과 배포

Skill + MCP + Hook + Agent 등을 하나의 설치 단위로 묶어서 팀 전체가 동일한 환경을 갖춘다

**개별 설치 (Plugin 없이)**

Skill A ← 수동 복사

Skill B ← 수동 복사

MCP 설정 ← 수동 설정

Hook 설정← 수동 설정

팀원 각자 반복...

**Plugin (한 번에)**

marketing-plugin

```
├── Skill A, B
├── MCP 설정
├── Hook 설정
└── Agent
```

**claude plugin add ← 한 줄이면 끝**

# 7개 기능의 관계

```
┌── CLAUDE.md ─────── Claude가 매번 읽는 규칙
│
├── Skill ──────────── 반복 업무를 레시피로 저장
│   └── MCP ─────────── Slack, Calendar 등 외부 도구 연결
│
├── Subagent ─────────── 하나의 작업을 백그라운드에서 처리
│   └── Agent Teams ─── 여러 Subagent가 동시에 작업
│
├── Hook ──────────── 특정 이벤트 발생 시 자동 실행
│
└── Plugin ─────────── 위의 모든 것을 묶어서 팀에 공유
```

Basics

# CLI 기초 — 이 5개면 충분하다

| 명령어 | 의미 | 비유 |
|---|---|---|
| cd 폴더명 | 폴더 이동 | 파인더에서 폴더 더블클릭 |
| ls | 목록 보기 | 폴더 열어서 뭐 있나 보기 |
| pwd | 현재 위치 | 주소창 보기 |
| cat 파일명 | 파일 내용 보기 | 메모장으로 파일 열어보기 |
| open 파일명 | 파일/폴더 열기 | 파인더에서 더블클릭 (Mac) |

**Claude Code가 나머지는 다 해준다. 명령어를 몰라도 말로 시키면 된다.**

Basics

# git + GitHub 기초

| 동작 | 비유 |
|---|---|
| git clone | Google Drive에서 폴더 다운로드 |
| git commit | 게임 세이브 |
| git branch | A안, B안 동시 작업 |
| git push | Google Drive에 업로드 |

**git = 실행 취소의 끝판왕. 뭘 해도 돌아갈 수 있다.**

| 개념 | 비유 |
|---|---|
| Pull Request (PR) | "이렇게 바꿨는데 괜찮아?" 리뷰 요청 |
| Merge | "오케이, 합치자" 리뷰 통과 후 반영 |

## 에디터 (선택사항)

Antigravity — 무료, AI 내장, 가장 간단

Cursor — AI 내장, CC 시너지

VSCode — 범용적, 확장 풍부

**"터미널에서 claude만 쳐도 충분합니다"**

# Quiz Time

| | | |
|---|---|---|
| **Quiz 3** | 현재 폴더 위치를 알려주는 명령어는? | **pwd** |
| **Quiz 4** | git에서 '게임 세이브'에 해당하는 동작은? | **git commit** |
| **Quiz 5** | '이렇게 바꿨는데 괜찮아?' 리뷰 요청은? | **Pull Request** |

# 모르면

# Claude에게 물어보면 된다.

**오늘 배운 것**

Setup → Experience → Why → 7개 핵심 기능 → Basics

내일은 더 근본적인 원리를 알게 되는 날입니다.

**수료가 아니라 시작이다.**